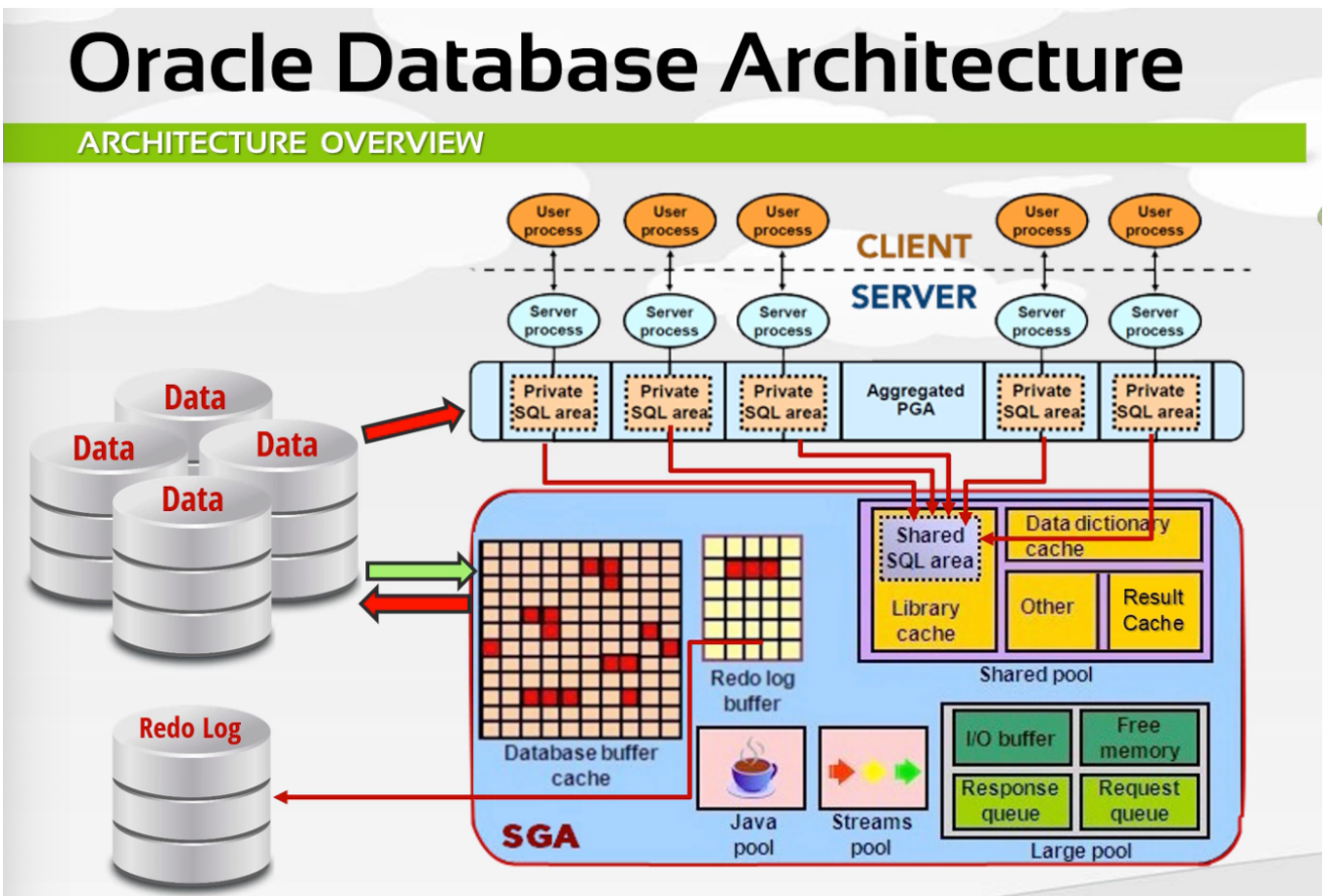


## Section3: Oracle Database Architecture

- SQL 튜닝을 잘하려면 **Oracle** 아키텍처를 최소 “기본 수준”은 알아야 한다.
- Oracle은 구조가 매우 크고 복잡하며(40년 이상 발전), 전부 다 알려면 DBA 수준이 필요하지만, 튜닝을 위해선 **핵심 구조(메모리/저장/동작 흐름)** 위주로 이해하면 충분하다.

### 1. 아키텍처



- 처리 흐름: **User Process → (Server) → Memory → Disk → Memory → User**
- 데이터는 디스크에 저장되지만, Oracle은 보통
  - 1) 디스크에서 **메모리**로 가져오고
  - 2) **정렬/조인** 같은 연산을 메모리에서 수행한 뒤
  - 3) 결과를 사용자에게 반환한다
- 그래서 튜닝 관점에서 핵심은 **메모리 동작** 이해이다.

## 2. 메모리의 2대 축: PGA vs SGA

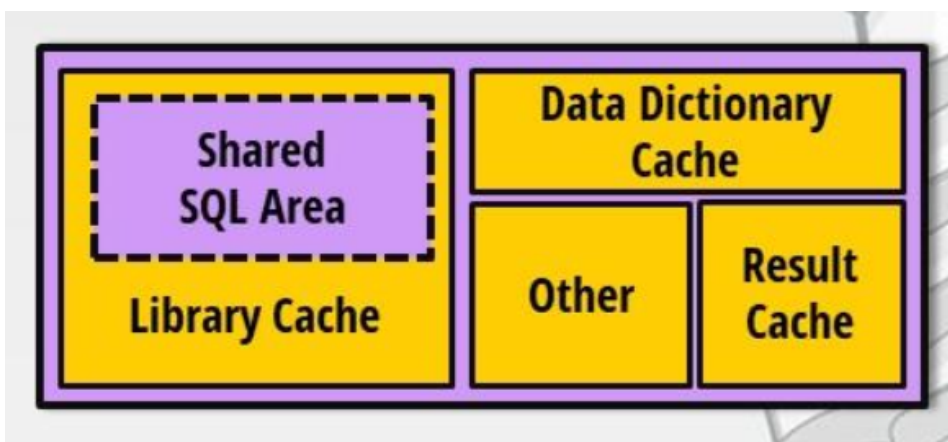
### 2.1 PGA (세션 전용)

- 사용자(세션)별로 따로 존재하는 private 메모리
- 세션 전용 정보/작업이 여기서 처리됨

### 2.2 SGA (공유)

- 모든 세션이 공유하는 메모리
- 여러 캐시(Shared Pool, Buffer Cache, Redo Log Buffer 등)로 나뉨
- 목적: 디스크 I/O를 줄여 성능 향상

## 3. Shared Pool



### 3.1 Data Dictionary Cache

- SQL 실행 전 필요한 객체 정의/권한/컬럼 정보 등을 저장
- 예) `SELECT * FROM employees` 실행 시  
테이블/컬럼 존재 여부, \*가 의미하는 컬럼 목록, 권한 등을 빠르게 확인

### 3.2 Library Cache (Shared SQL Area의 컨테이너)

- SQL 실행에 필요한 **Shared SQL Area**(파싱/실행계획 등 정보)를 보관
- 실행계획 생성은 비용이 큰 작업이므로,  
같은/유사 SQL이 다시 실행될 때 기존 실행계획 재사용으로 성능 향상

- 메모리 부족 시: 덜 쓰는 **SQL Area**를 제거하고 재배치/확장 등으로 관리

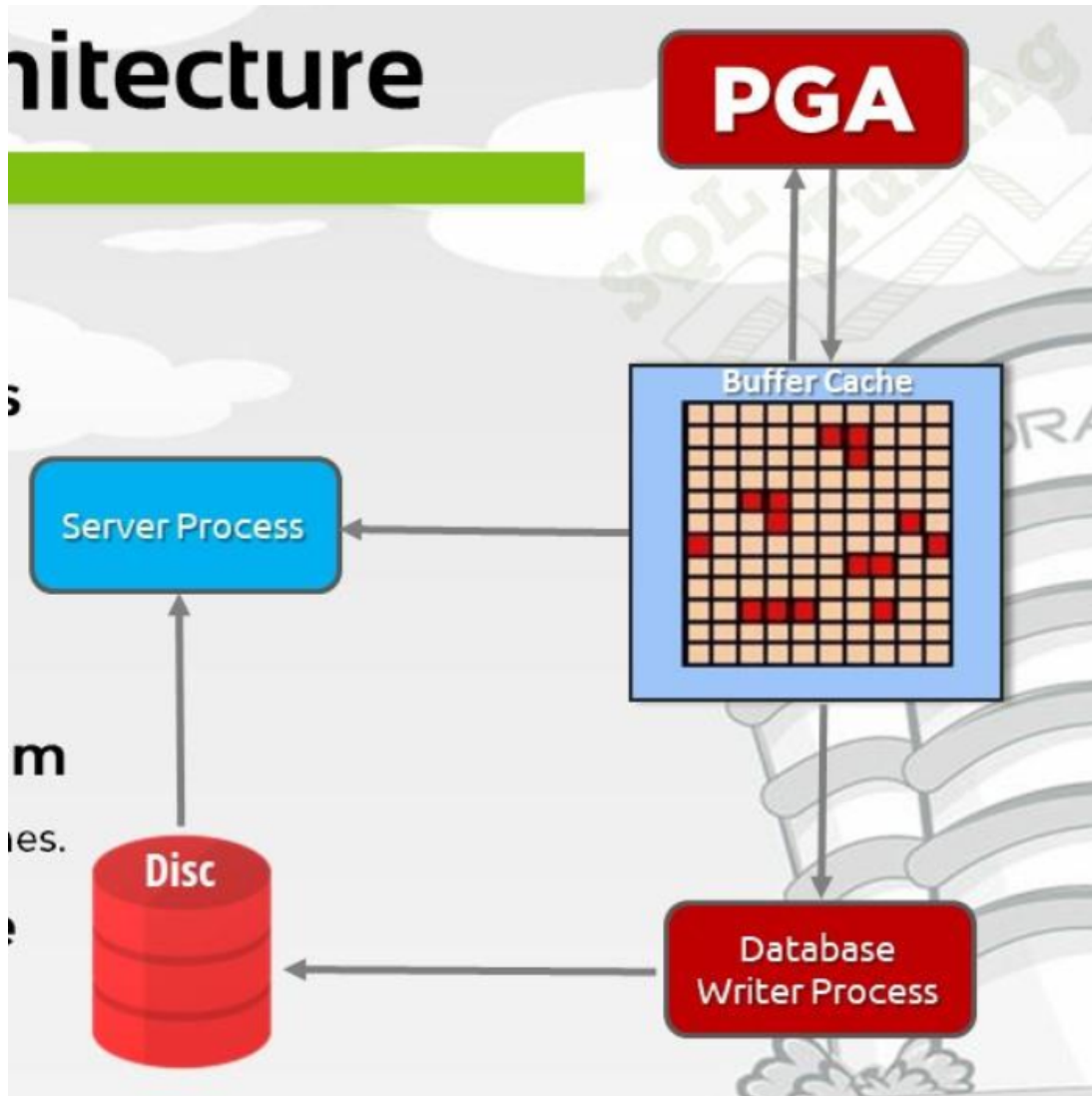
### 3.3 Result Cache

- 반복 실행되는 쿼리 및 함수의 결과를 저장해 다음 호출에 즉시 반환
- 원본 데이터가 변경되면 관련 캐시 결과는 **무효화(Flush)**되어 재계산됨
- 모든 결과를 저장하면 메모리 문제가 생기므로 **일부(자주 쓰는 것)만 저장**

### 3.4 Shared Pool 처리 흐름 요약

- Dictionary Cache로 **SQL 유효성/권한/정의 확인**
  - Library Cache로 **기존 실행계획 존재 여부 확인**
  - Result Cache로 **결과 존재 여부 확인**
  - 없으면 실행계획 생성 후 디스크/버퍼캐시 접근
- 

## 4. Buffer Cache

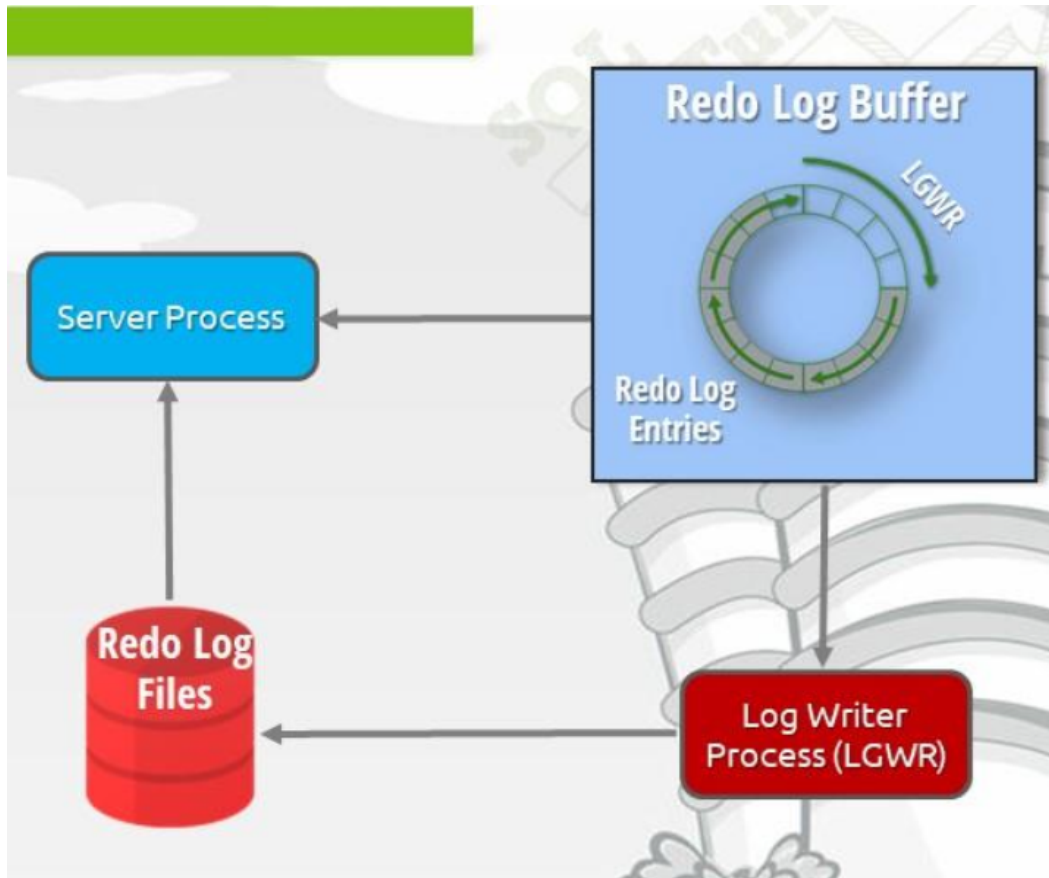


- SGA에서 가장 큰 영역
- 디스크에서 읽은 **데이터 블록(테이블/인덱스 블록)**의 복사본을 저장
- 서버는 디스크를 보기 전에 **Buffer Cache**를 먼저 확인
  - 전부 있으면 캐시에서 바로 읽음
  - 일부만 있으면 없는 블록만 디스크에서 읽음
  - 하나도 없으면 필요한 블록을 디스크에서 읽어 캐시에 적재
- 캐시 크기는 한정 → 복잡한 알고리즘으로  
자주 쓰는/최근 쓰는 블록을 유지, 덜 쓰는 것은 제거

## 4.1 Dirty Block

- DML(INSERT/UPDATE/DELETE)로 변경된 블록은 Buffer Cache에서 먼저 바뀌고  
변경된 블록을 **Dirty Block**이라 부름
- 디스크 반영은 **DB Writer 프로세스**가 수행(커밋 후 반영 개념)

## 5. Redo Log Buffer 핵심

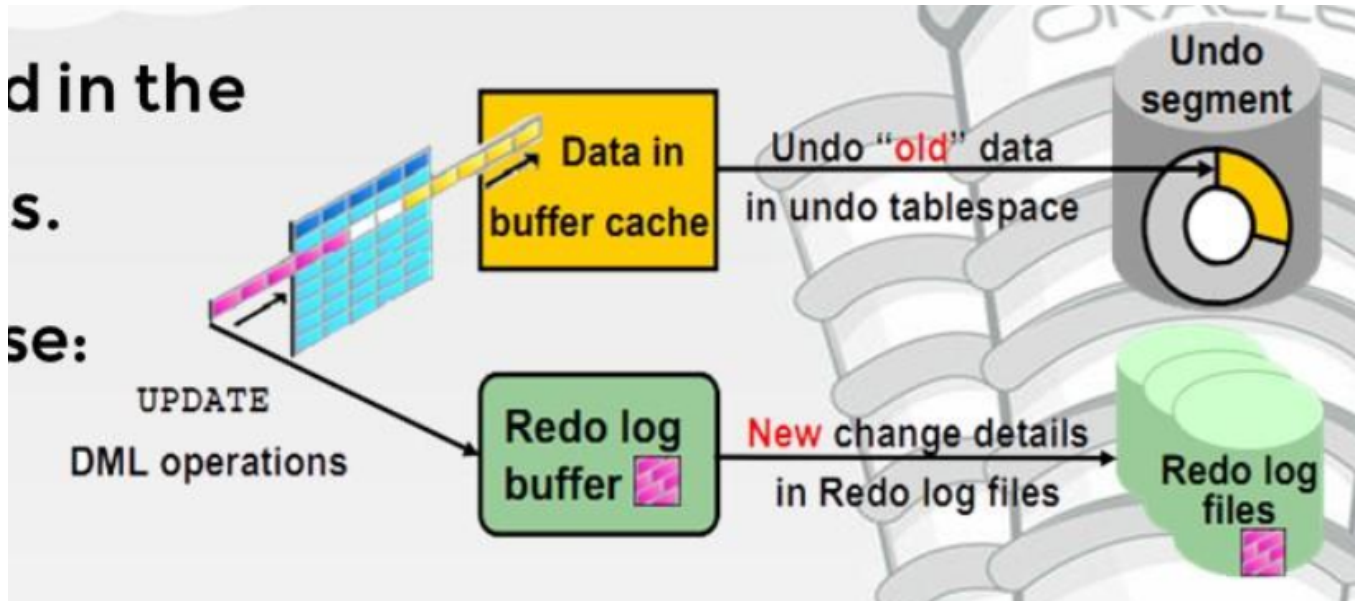


- SGA의 메모리 영역
- DML/DDL로 발생한 변경에 대해 **Redo Log Entry** 생성
- Redo Log Entry는 복구(Recovery)를 위한 변경 기록
- Redo Log Buffer에 잠시 저장되었다가 **LGWR(Log Writer)**가 **Online Redo Log File(디스크)**로 지속적으로 기록
- 버퍼는 유한하므로 **Circular Buffer** 방식

Redo는 “복구” 용도이며, 롤백 메커니즘은 Undo

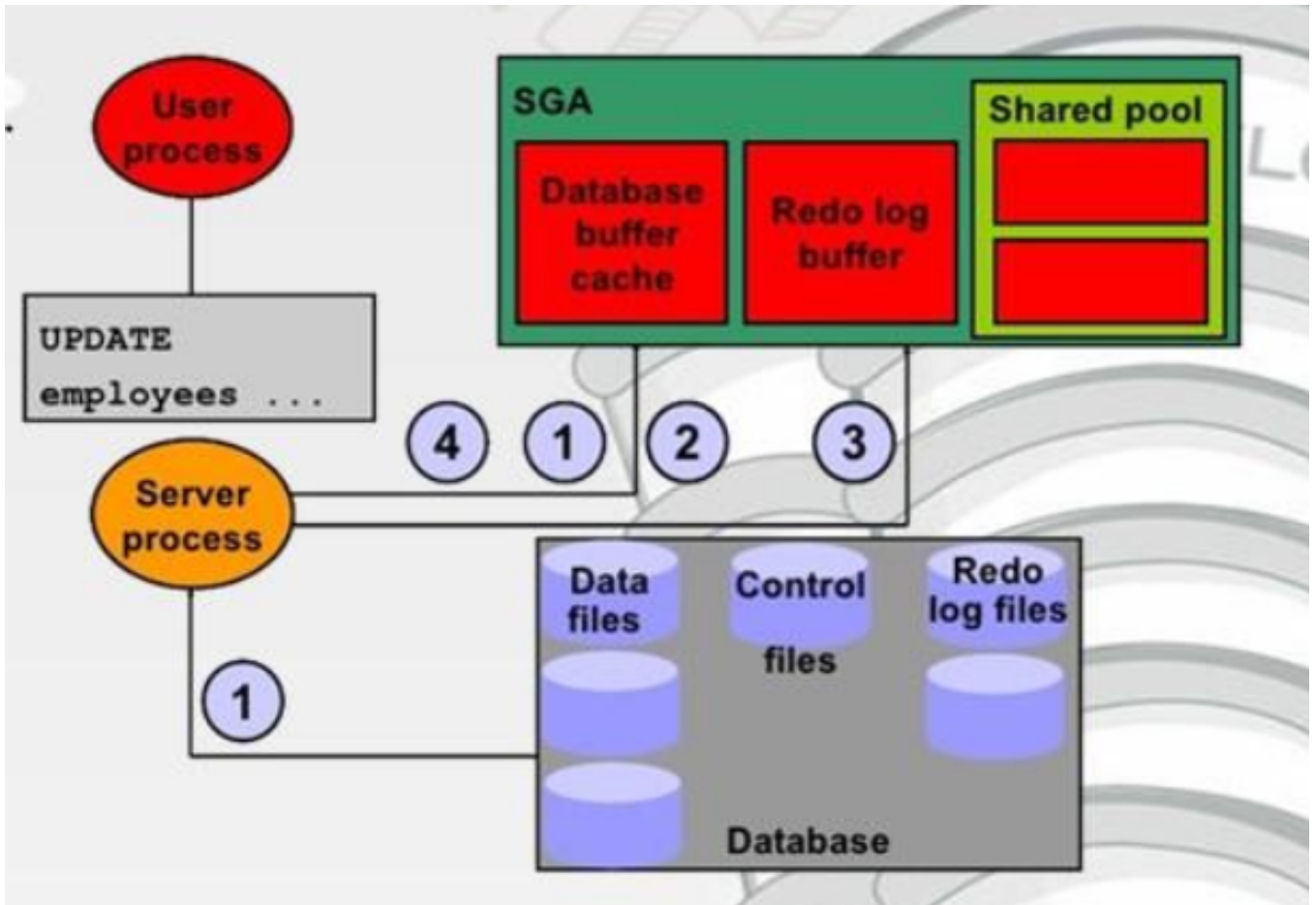
튜닝 관점에서는 “변경마다 redo 비용이 발생한다”는 점이 중요

## 6. Undo 핵심



- 트랜잭션이 데이터를 변경할 때, 변경 전 원본을 복사해 저장한 것이 **Undo Data**
- Undo의 목적
  - 1) **Rollback**(변경 취소)
  - 2) **Read Consistency**(커밋 전 다른 사용자는 원래 값 보게 함)
  - 3) **Flashback**(특정 시점 복구/변경 추적 기능 지원)
- 커밋 후에도 한동안 Undo가 남아:
  - 긴 쿼리의 일관된 읽기 지원
  - Flashback 지원

## 7. DML 처리 & COMMIT 흐름

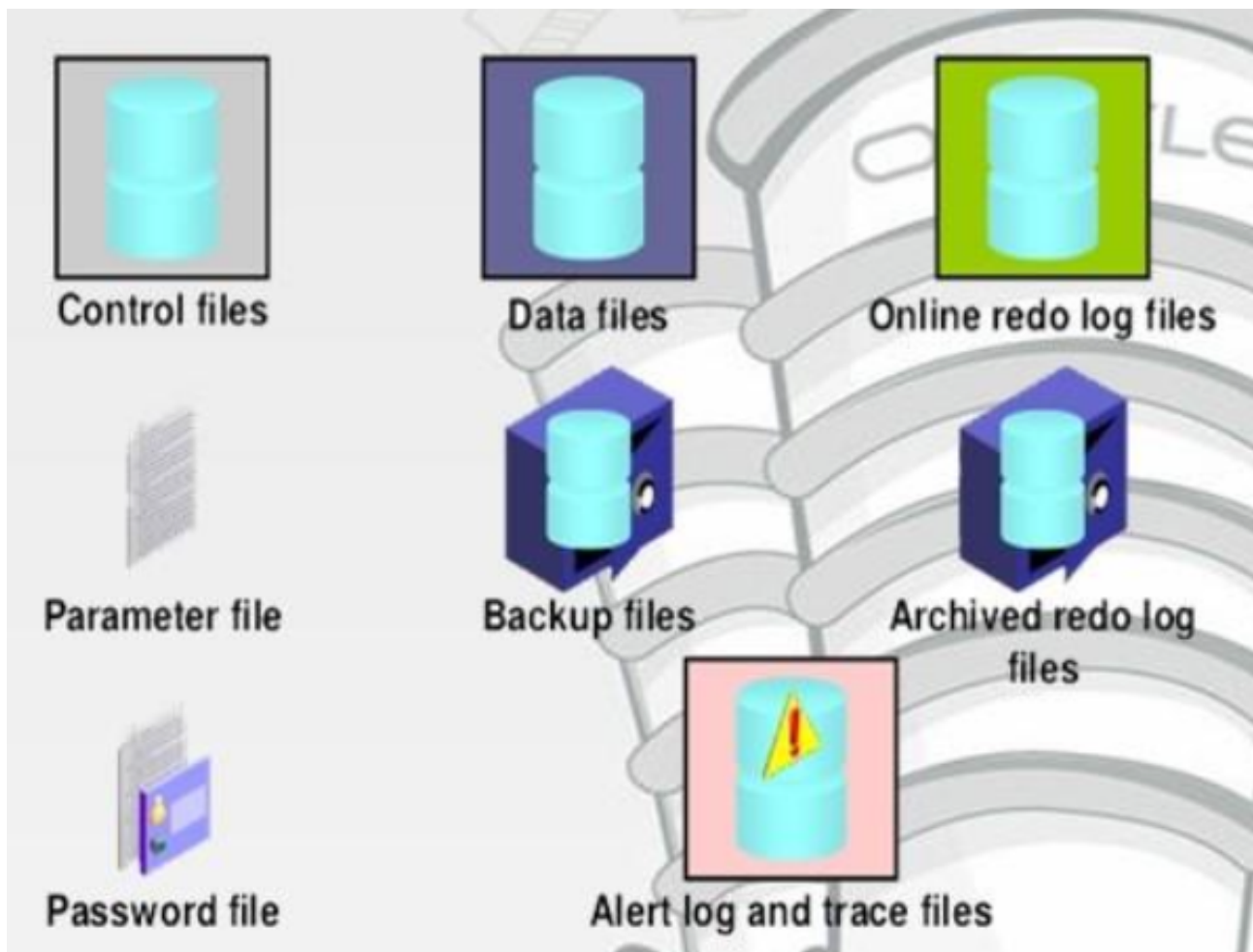


- 1) Library Cache(Shared SQL Area) 확인: 유사 SQL 존재하면 재파싱 감소
- 2) Data Dictionary Cache 확인: 객체 정의/권한 정보 재사용
- 3) Buffer Cache 확인:
  - 대상 데이터/Undo 관련 블록이 없으면 디스크에서 읽어 캐시에 적재
- 4) 대상 블록 **Lock** (다른 사용자 변경 방지)
- 5) 변경은 **Buffer Cache**에서 수행 → 변경 블록은 Dirty Block
- 6) 변경 전에 Redo Log Buffer에 redo 기록 생성(**Write-ahead logging**)
- 7) 사용자에게 영향받은 행 수 반환(트랜잭션은 계속될 수 있음)
- 8) COMMIT 시:
  - 커밋 레코드 생성
  - LGWR가 Redo Log Buffer 내용을 Online Redo Log File로 기록
  - DBWR가 Dirty Block을 디스크로 기록
  - Lock 해제
  - 트랜잭션 완료 응답

## 8. Automatic Memory Management(자동 메모리 관리)

- 각 메모리 영역 크기는 성능에 큰 영향을 줌
  - 수동 관리가 어렵기 때문에 Oracle은 **Automatic Memory Management** 기능 제공
  - SGA/PGA 사이(또는 내부)에서 필요에 따라 메모리를 자동으로 재분배
  - 보통 자동 메모리 관리를 켜두는 것이 성능/안정성에 유리
    - out of memory 오류 방지에도 도움
- 

## 9. Database Storage(디스크 파일)

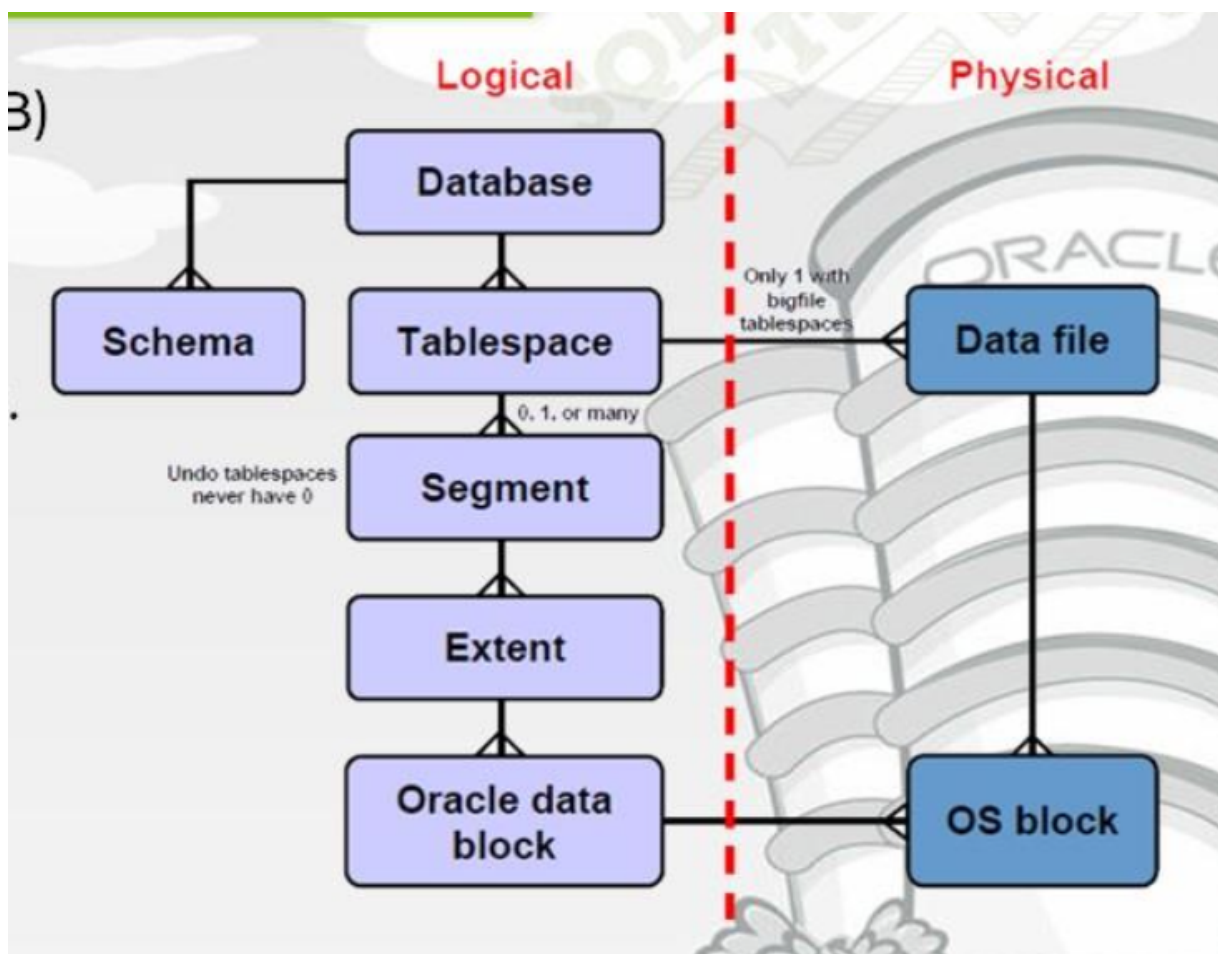


Oracle의 “스토리지”는 디스크 파일을 의미(메모리는 스토리지로 보지 않음).

- **Control File:** 물리적 DB 구조 정보(없으면 접근 불가, 매우 중요)
- **Data Files:** 실제 데이터(테이블/프로시저/앱 데이터 등) 저장
- **Online Redo Log Files:** 복구용 redo 기록 저장(빠르게 유지 필요)

- **Archived Redo Log Files:** online redo의 내용을 옮겨 저장(공간 확보/복구 지원)
- **Backup Files:** 데이터 파일의 복사본(재해 복구/복원)
- **Parameter File:** 인스턴스 설정 파라미터
- **Password File:** sysdba/sysoper/sysasm 등 관리자 계정 인증 정보
- **Alert Log / Trace Files:** 오류/메시지 로그 및 추적 정보

## 10. 논리적 저장 구조(Logical)



- **Block:** 최소 논리 단위(2KB~32KB, 보통 8KB)
- **Extent:** 여러 연속 블록의 묶음
- **Segment:** 여러 Extent의 묶음
  - 종류: Data / Index / Undo / Temporary Segment
- **Tablespace:** 여러 Segment의 묶음(가장 큰 논리 단위)
  - 유형: Temporary Tablespace(임시, tempfile) / Permanent Tablespace(영구, datafile)
  - 필수: SYSTEM, SYSAUX

- 사용자마다 기본 Permanent Tablespace가 할당됨(해당 스키마 오브젝트 저장 위치)
  - 운영 측면에서: 온라인/오프라인, 백업/복구, import/export, transport 등 단위 작업 가능
  - **Schema**: 사용자 소유의 DB 오브젝트(테이블/뷰/프로시저 등) 집합
- 

## 11. Database Block

- Oracle은 데이터를 **블록** 단위로 저장/읽음(디스크/메모리 모두)
- 블록은:
  - **Block Header**(블록 타입/테이블 정보/Row Directory)
  - **Row Data + Free Space**
- Row Directory가 가리키는 주소가 **ROWID**
  - ROWID로 행의 위치를 빠르게 접근

### 11.1 Free Space & PCTFREE

- UPDATE로 행 크기가 커질 수 있으므로 블록에 여유 공간을 남김
- 여유 공간이 없으면:
  - 행 이동 가능성 증가 → I/O 증가 → 성능 저하
  - 행 주소 변경으로 인덱스 등 추가 작업 발생 가능
- 테이블 생성 시 `PCTFREE` / `PCTUSED`로 여유 공간 정책 조정 가능

### 11.2 튜닝 포인트

- 성능은 “얼마나 적은 블록을 읽는가”와 직결
- 블록 읽기를 줄이면 성능이 크게 향상될 수 있음